

Image features exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the [assignments page](#) on the course website.

We have seen that we can achieve reasonable performance on an image classification task by training a linear classifier on the pixels of the input image. In this exercise we will show that we can improve our classification performance by training linear classifiers not on raw pixels but on features that are computed from the raw pixels.

All of your work for this exercise will be done in this notebook.

```
In [23]: # Update ipython version (requires restart)
!pip install ipython==8.1.0
```

```
In [24]: ### IF YOU ARE USING COLAB, UNCOMMENT AND RUN THIS BLOCK FIRST ###

# # Mount google drive to allow access to your files
# from google.colab import drive
# drive.mount('/content/drive')
# drive_folder = '/content/drive/MyDrive'
# # Adjust this line to be the assignment1 folder in your google drive
# notebook_folder = drive_folder + '/cs682/assignment1'
# %cd {notebook_folder}
```

```
In [25]: from __future__ import print_function
import random
import numpy as np
from cs682.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

The autoreload extension is already loaded. To reload it, use:

```
%reload_ext autoreload
```

Load data

Similar to previous exercises, we will load CIFAR-10 data from disk.

```
In [26]: from cs682.features import color_histogram_hsv, hog_feature

def get_CIFAR10_data(num_training=49000, num_validation=1000, num_test=1000)
    # Load the raw CIFAR-10 data
    cifar10_dir = 'cs682/datasets/cifar-10-batches-py'

    # Don't forget to run get_datasets.sh, or this will throw an error
    X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

    # Subsample the data
    mask = list(range(num_training, num_training + num_validation))
    X_val = X_train[mask]
    y_val = y_train[mask]
    mask = list(range(num_training))
    X_train = X_train[mask]
    y_train = y_train[mask]
    mask = list(range(num_test))
    X_test = X_test[mask]
    y_test = y_test[mask]

    return X_train, y_train, X_val, y_val, X_test, y_test

# Cleaning up variables to prevent loading data multiple times (which may ca
try:
    del X_train, y_train
    del X_test, y_test
    print('Clear previously loaded data.')
except:
    pass

X_train, y_train, X_val, y_val, X_test, y_test = get_CIFAR10_data()
```

Clear previously loaded data.

Extract Features

For each image we will compute a Histogram of Oriented Gradients (HOG) as well as a color histogram using the hue channel in HSV color space. We form our final feature vector for each image by concatenating the HOG and color histogram feature vectors.

Roughly speaking, HOG should capture the texture of the image while ignoring color information, and the color histogram represents the color of the input image while ignoring texture. As a result, we expect that using both together ought to work better than using either alone. Verifying this assumption would be a good thing to try for your interests.

The `hog_feature` and `color_histogram_hsv` functions both operate on a single image and return a feature vector for that image. The `extract_features` function takes a set of images and a list of feature functions and evaluates each feature function on each

image, storing the results in a matrix where each column is the concatenation of all feature vectors for a single image.

```
In [27]: from cs682.features import *

num_color_bins = 10 # Number of bins in the color histogram
feature_fns = [hog_feature, lambda img: color_histogram_hsv(img, nbin=num_color_bins)]
X_train_feats = extract_features(X_train, feature_fns, verbose=True)
X_val_feats = extract_features(X_val, feature_fns)
X_test_feats = extract_features(X_test, feature_fns)

# Preprocessing: Subtract the mean feature
mean_feat = np.mean(X_train_feats, axis=0, keepdims=True)
X_train_feats -= mean_feat
X_val_feats -= mean_feat
X_test_feats -= mean_feat

# Preprocessing: Divide by standard deviation. This ensures that each feature
# has roughly the same scale.
std_feat = np.std(X_train_feats, axis=0, keepdims=True)
X_train_feats /= std_feat
X_val_feats /= std_feat
X_test_feats /= std_feat

# Preprocessing: Add a bias dimension
X_train_feats = np.hstack([X_train_feats, np.ones((X_train_feats.shape[0], 1))])
X_val_feats = np.hstack([X_val_feats, np.ones((X_val_feats.shape[0], 1))])
X_test_feats = np.hstack([X_test_feats, np.ones((X_test_feats.shape[0], 1))])
```

Done extracting features for 1000 / 49000 images
Done extracting features for 2000 / 49000 images
Done extracting features for 3000 / 49000 images
Done extracting features for 4000 / 49000 images
Done extracting features for 5000 / 49000 images
Done extracting features for 6000 / 49000 images
Done extracting features for 7000 / 49000 images
Done extracting features for 8000 / 49000 images
Done extracting features for 9000 / 49000 images
Done extracting features for 10000 / 49000 images
Done extracting features for 11000 / 49000 images
Done extracting features for 12000 / 49000 images
Done extracting features for 13000 / 49000 images
Done extracting features for 14000 / 49000 images
Done extracting features for 15000 / 49000 images
Done extracting features for 16000 / 49000 images
Done extracting features for 17000 / 49000 images
Done extracting features for 18000 / 49000 images
Done extracting features for 19000 / 49000 images
Done extracting features for 20000 / 49000 images
Done extracting features for 21000 / 49000 images
Done extracting features for 22000 / 49000 images
Done extracting features for 23000 / 49000 images
Done extracting features for 24000 / 49000 images
Done extracting features for 25000 / 49000 images
Done extracting features for 26000 / 49000 images
Done extracting features for 27000 / 49000 images
Done extracting features for 28000 / 49000 images
Done extracting features for 29000 / 49000 images
Done extracting features for 30000 / 49000 images
Done extracting features for 31000 / 49000 images
Done extracting features for 32000 / 49000 images
Done extracting features for 33000 / 49000 images
Done extracting features for 34000 / 49000 images
Done extracting features for 42000 / 49000 images
Done extracting features for 43000 / 49000 images
Done extracting features for 44000 / 49000 images
Done extracting features for 45000 / 49000 images
Done extracting features for 46000 / 49000 images
Done extracting features for 35000 / 49000 images
Done extracting features for 36000 / 49000 images
Done extracting features for 37000 / 49000 images
Done extracting features for 38000 / 49000 images
Done extracting features for 39000 / 49000 images
Done extracting features for 40000 / 49000 images
Done extracting features for 41000 / 49000 images
Done extracting features for 47000 / 49000 images
Done extracting features for 48000 / 49000 images

Train SVM on features

Using the multiclass SVM code developed earlier in the assignment, train SVMs on top of the features extracted above; this should achieve better results than training SVMs directly on top of raw pixels.

```
In [28]: # Use the validation set to tune the learning rate and regularization strength

from cs682.classifiers.linear_classifier import LinearSVM

learning_rates = [1e-9, 1e-8, 1e-7]
regularization_strengths = [5e4, 5e5, 5e6]

results = {}
best_val = -1
best_svm = None

#####
# TODO:
# Use the validation set to set the learning rate and regularization strength
# This should be identical to the validation that you did for the SVM; save
# the best trained classifier in best_svm. You might also want to play
# with different numbers of bins in the color histogram. If you are careful
# you should be able to get accuracy of near 0.44 on the validation set.
#####
# The provided values over-regularize these standardized 154-d features (even
# pair with lr * reg = 5e-3 collapses W to the same class-mean template, ~0.
# Larger learning rates with much smaller regularization are searched as well
# Pairs with lr * reg >= 1 are skipped because training diverges.
search_lrs = learning_rates + [1e-3, 3e-3, 1e-2, 3e-2]
search_regs = regularization_strengths + [0, 1e-3, 1e-2, 1e-1]

for lr in search_lrs:
    for reg in search_regs:
        if lr * reg >= 1:
            continue
        svm = LinearSVM()
        svm.train(X_train_feats, y_train, learning_rate=lr, reg=reg,
                  num_iters=3000, verbose=False)
        train_accuracy = np.mean(svm.predict(X_train_feats) == y_train)
        val_accuracy = np.mean(svm.predict(X_val_feats) == y_val)
        results[(lr, reg)] = (train_accuracy, val_accuracy)
        if val_accuracy > best_val:
            best_val = val_accuracy
            best_svm = svm

#####
#                               END OF YOUR CODE
#####

# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
```

```
lr, reg, train_accuracy, val_accuracy))  
print('best validation accuracy achieved during cross-validation: %f' % best
```

```
lr 1.000000e-09 reg 0.000000e+00 train accuracy: 0.122388 val accuracy: 0.113000
lr 1.000000e-09 reg 1.000000e-03 train accuracy: 0.084082 val accuracy: 0.086000
lr 1.000000e-09 reg 1.000000e-02 train accuracy: 0.126490 val accuracy: 0.109000
lr 1.000000e-09 reg 1.000000e-01 train accuracy: 0.090061 val accuracy: 0.090000
lr 1.000000e-09 reg 5.000000e+04 train accuracy: 0.105102 val accuracy: 0.106000
lr 1.000000e-09 reg 5.000000e+05 train accuracy: 0.122857 val accuracy: 0.110000
lr 1.000000e-09 reg 5.000000e+06 train accuracy: 0.416857 val accuracy: 0.409000
lr 1.000000e-08 reg 0.000000e+00 train accuracy: 0.095653 val accuracy: 0.101000
lr 1.000000e-08 reg 1.000000e-03 train accuracy: 0.137184 val accuracy: 0.131000
lr 1.000000e-08 reg 1.000000e-02 train accuracy: 0.104102 val accuracy: 0.113000
lr 1.000000e-08 reg 1.000000e-01 train accuracy: 0.113347 val accuracy: 0.106000
lr 1.000000e-08 reg 5.000000e+04 train accuracy: 0.123204 val accuracy: 0.107000
lr 1.000000e-08 reg 5.000000e+05 train accuracy: 0.415898 val accuracy: 0.417000
lr 1.000000e-08 reg 5.000000e+06 train accuracy: 0.401714 val accuracy: 0.401000
lr 1.000000e-07 reg 0.000000e+00 train accuracy: 0.154592 val accuracy: 0.145000
lr 1.000000e-07 reg 1.000000e-03 train accuracy: 0.153000 val accuracy: 0.135000
lr 1.000000e-07 reg 1.000000e-02 train accuracy: 0.142816 val accuracy: 0.140000
lr 1.000000e-07 reg 1.000000e-01 train accuracy: 0.126429 val accuracy: 0.145000
lr 1.000000e-07 reg 5.000000e+04 train accuracy: 0.416776 val accuracy: 0.417000
lr 1.000000e-07 reg 5.000000e+05 train accuracy: 0.404857 val accuracy: 0.399000
lr 1.000000e-07 reg 5.000000e+06 train accuracy: 0.318980 val accuracy: 0.307000
lr 1.000000e-03 reg 0.000000e+00 train accuracy: 0.507367 val accuracy: 0.496000
lr 1.000000e-03 reg 1.000000e-03 train accuracy: 0.508449 val accuracy: 0.495000
lr 1.000000e-03 reg 1.000000e-02 train accuracy: 0.508347 val accuracy: 0.497000
lr 1.000000e-03 reg 1.000000e-01 train accuracy: 0.505102 val accuracy: 0.494000
lr 3.000000e-03 reg 0.000000e+00 train accuracy: 0.511388 val accuracy: 0.502000
lr 3.000000e-03 reg 1.000000e-03 train accuracy: 0.511265 val accuracy: 0.506000
lr 3.000000e-03 reg 1.000000e-02 train accuracy: 0.514082 val accuracy: 0.498000
```

```

lr 3.000000e-03 reg 1.000000e-01 train accuracy: 0.507163 val accuracy: 0.486000
lr 1.000000e-02 reg 0.000000e+00 train accuracy: 0.509082 val accuracy: 0.489000
lr 1.000000e-02 reg 1.000000e-03 train accuracy: 0.511204 val accuracy: 0.496000
lr 1.000000e-02 reg 1.000000e-02 train accuracy: 0.508694 val accuracy: 0.494000
lr 1.000000e-02 reg 1.000000e-01 train accuracy: 0.501143 val accuracy: 0.494000
lr 3.000000e-02 reg 0.000000e+00 train accuracy: 0.503020 val accuracy: 0.493000
lr 3.000000e-02 reg 1.000000e-03 train accuracy: 0.501959 val accuracy: 0.483000
lr 3.000000e-02 reg 1.000000e-02 train accuracy: 0.507020 val accuracy: 0.498000
lr 3.000000e-02 reg 1.000000e-01 train accuracy: 0.495245 val accuracy: 0.496000
best validation accuracy achieved during cross-validation: 0.506000

```

```

In [29]: # Evaluate your trained SVM on the test set
y_test_pred = best_svm.predict(X_test_feats)
test_accuracy = np.mean(y_test == y_test_pred)
print(test_accuracy)

```

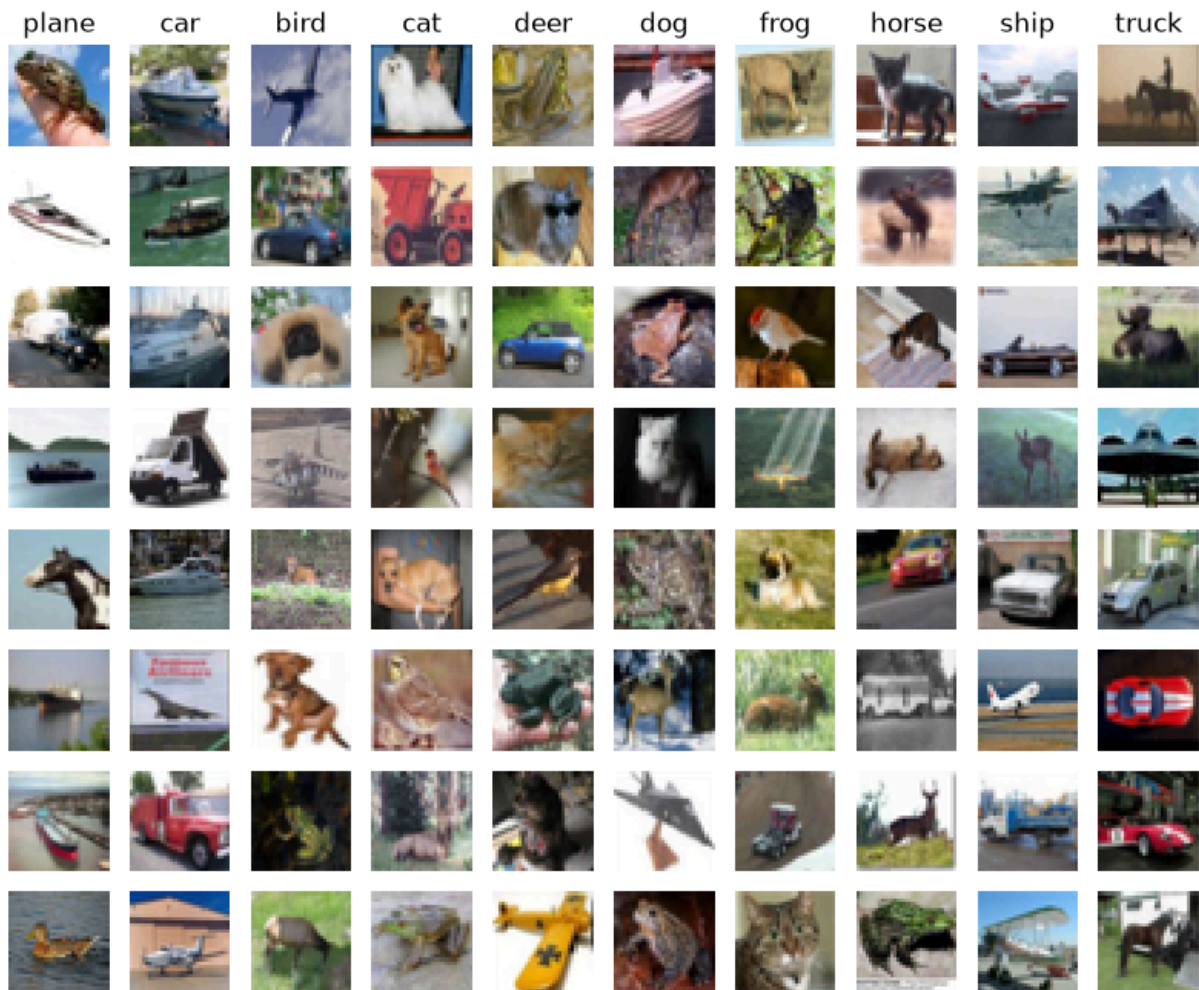
0.492

```

In [30]: # An important way to gain intuition about how an algorithm works is to
# visualize the mistakes that it makes. In this visualization, we show examples
# of images that are misclassified by our current system. The first column
# shows images that our system labeled as "plane" but whose true label is
# something other than "plane".

examples_per_class = 8
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
for cls, cls_name in enumerate(classes):
    idxs = np.where((y_test != cls) & (y_test_pred == cls))[0]
    idxs = np.random.choice(idxs, examples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt.subplot(examples_per_class, len(classes), i * len(classes) + cls)
        plt.imshow(X_test[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls_name)
plt.show()

```



Inline question 1:

Describe the misclassification results that you see. Do they make sense?

Answer: Many of the mistakes make sense given what the features capture. HOG describes coarse edge orientations (shape and outline) and the hue histogram describes the overall color distribution, not what the object actually is. In my results:

- *Plane* and *ship* are confused both ways: the *plane* column is mostly boats and ships, and the *ship* column is mostly planes. Both usually sit on a plain blue or grey background (sky or water) that dominates the color histogram, and both have long horizontal outlines.
- Vehicles are confused with each other: the *car* column contains several boats, trucks (including a dump truck and a red fire truck) and planes, and the *truck* column contains several cars. They share boxy, straight-edged shapes.
- Animals are confused with each other: the *cat* column is mostly dogs, the *horse* column contains cats and dogs, and the *bird*, *deer*, *dog* and *frog* columns mix deer, cats, dogs, frogs and birds. They have similar outlines and brown or green natural backgrounds.

- Some errors don't make intuitive sense, such as horses (one with a rider) labeled *truck*, a yellow plane labeled *deer*, or a red truck labeled *cat*. These images have dominant edges or colors that happen to match another class's template, which a linear classifier on hand-crafted features can't separate.

Overall, the misclassifications mostly reflect visual similarity at the level of these features, which is why the neural network below does better.

Neural Network on image features

Earlier in this assignment we saw that training a two-layer neural network on raw pixels achieved better classification performance than linear classifiers on raw pixels. In this notebook we have seen that linear classifiers on image features outperform linear classifiers on raw pixels.

For completeness, we should also try training a neural network on image features. This approach should outperform all previous approaches: you should easily be able to achieve over 55% classification accuracy on the test set; our best model achieves about 60% classification accuracy.

```
In [31]: # Preprocessing: Remove the bias dimension
# Make sure to run this cell only ONCE
print(X_train_feats.shape)
X_train_feats = X_train_feats[:, :-1]
X_val_feats = X_val_feats[:, :-1]
X_test_feats = X_test_feats[:, :-1]

print(X_train_feats.shape)
```

```
(49000, 155)
```

```
(49000, 154)
```

```
In [32]: from cs682.classifiers.neural_net import TwoLayerNet

input_dim = X_train_feats.shape[1]
hidden_dim = 500
num_classes = 10

net = TwoLayerNet(input_dim, hidden_dim, num_classes)
best_net = None

#####
# TODO: Train a two-layer neural network on image features. You may want to
# cross-validate various parameters as in previous sections. Store your best
# model in the best_net variable.
#####
# Feature vectors are small (~154-d) and normalised, so much larger learning
# rates than on raw pixels are appropriate.
best_val = -1
results = {}
learning_rates = [1e-1, 3e-1, 5e-1]
```

```

regs = [1e-4, 1e-3, 5e-3]

for lr in learning_rates:
    for reg in regs:
        net = TwoLayerNet(input_dim, hidden_dim, num_classes)
        net.train(X_train_feats, y_train, X_val_feats, y_val,
                  num_iters=2000, batch_size=200,
                  learning_rate=lr, learning_rate_decay=0.95,
                  reg=reg, verbose=False)
        train_acc = (net.predict(X_train_feats) == y_train).mean()
        val_acc = (net.predict(X_val_feats) == y_val).mean()
        results[(lr, reg)] = (train_acc, val_acc)
        print('lr %.1e reg %.1e -> train %.3f val %.3f' % (lr, reg, train_acc, val_acc))
        if val_acc > best_val:
            best_val = val_acc
            best_net = net

print('best validation accuracy: %f' % best_val)
#####
#                                     END OF YOUR CODE
#####

```

```

lr 1.0e-01 reg 1.0e-04 -> train 0.558 val 0.545
lr 1.0e-01 reg 1.0e-03 -> train 0.555 val 0.534
lr 1.0e-01 reg 5.0e-03 -> train 0.535 val 0.521
lr 3.0e-01 reg 1.0e-04 -> train 0.670 val 0.583
lr 3.0e-01 reg 1.0e-03 -> train 0.646 val 0.602
lr 3.0e-01 reg 5.0e-03 -> train 0.566 val 0.540
lr 5.0e-01 reg 1.0e-04 -> train 0.724 val 0.594
lr 5.0e-01 reg 1.0e-03 -> train 0.672 val 0.577
lr 5.0e-01 reg 5.0e-03 -> train 0.574 val 0.555
best validation accuracy: 0.602000

```

In [33]: *# Run your best neural net classifier on the test set. You should be able to get more than 55% accuracy.*

```

test_acc = (best_net.predict(X_test_feats) == y_test).mean()
print(test_acc)

```

0.57